

ebaleytherogue's No Man's Sky Bases Report Generator Script

[What is This?](#)

[What this Isn't](#)

[Using the Report Files](#)

[Using These Files to Create Reports](#)

[Prerequisites](#)

[Steps](#)

[Tips](#)

[Base-Names in Reports Might be Out-of-Date if Not Uploaded](#)

[Change Log](#)

[COSMOS UPDATE - More Base Types Added & Useless Reports Removed](#)

[What Cosmos added to your save](#)

[The first thing I changed: report BaseType](#)

[Cosmos has changed the significance of PlanetIndex 0](#)

[I found one actual matching problem](#)

[One more useful Cosmos distinction](#)

[Bottom line](#)

[Major Update - Extraneous Bases That Were Listed from Temporary Data Removed from results.](#)

[More Date Fields Added to the Reports](#)

[Duplicate Bases Falsely Inflating the Base Count](#)

[On-Planet Locations Added](#)

[Changed Reports Folder to always be "nms_base_reports"](#)

[Changed Main Script Name to NOT Include Version Number and .BAT file to Reference This New Name](#)

[Changed the Called JSON Filename to save.hg.json from save8.hg.json](#)

What is This?

This is a Python script that analyzes one's gamesave file's JSON export to generate a list of player bases. As a bonus I added the portal location for each base to the data, making it simple to share any base's location, that you wish, with other players. Thus, with this data you can:

- Know how many bases you have (and how many in each specific galaxy)
- Quickly share your base location with others without having to manually type the addresses into chat or journey to the base and take screenshots

To create the reports I heavily leaned on the JSON file's Teleporter entries. This got me what I needed. I attempted to give information about how many base objects exist, within any given base, but I could not get consistently accurate counts, because that information is scattered throughout the JSON, for any given base, and I did not want to provide inaccurate counts. I felt nothing is better than something in that regard.

Frankly, the hardest part of this script was calculating the bases' portal addresses hex values to compute properly. I'm not a math guy, but with AI's help I got it done. I used <https://nmscd.com/NMSGlyphRandomizer/?ref=nmscd> pasting the 12 digit hex addresses into that site's 'Enter glyphs' field to visually check my values against the portal glyphs, and I achieved a 100% success rate with what I saw in Camera Mode whilst on planet.

For you, this means you could copy and paste any value, from the reports' "Glyph String (No Spaces)" column, into that web site's field, and get the visual glyphs for that base.
Easy peasy.

Feel free to share this script, change this script, do whatever you like with this script. It was an exercise for me, kind of a side-quest in the game itself. Have fun with it!

I will keep it in the [ebaleytherogue's No Man's Sky_All Your Bases via Python Script_SHA...](#) shared folder unless I end up moving it to GitHub or something, although if I do, I'll drop a note in the folder with that location.

What this Isn't

This is not a gamesave editor. It does not manipulate your gamesave file in anyway. It simply interrogates a JSON export and produces report files.

Using the Report Files

The report files are simply csv (comma separated values) formatted text files that you could read in a text editor. However, Google Sheets, Excel, Libre Office, etc., know to read CSV files as spreadsheets. So, simply drag the .csv files to a folder in your Google Docs, and then open them. You can then view/sort/manipulate them as you would any spreadsheet.

Using These Files to Create Reports

Prerequisites

In Windows 11 you will need Python capability and PowerShell capability to run it. I suppose you could just use a good old Windows command line window if you'd rather, but I used PowerShell. This document is not a PowerShell nor Python tutorial, so follow your googles and AI's and you'll get there. All you have to do is enable these things and/or perform a small installation to get them up and running. You do not need to program anything! At worst you'll need to right-click-open the .bat file to change the name of your JSON export.

Steps

Extract the files to a folder in your Windows system. I suggest something like shown below, but it can be anywhere.

This PC > OS (C:) > Apps > NMSJSONEXPORTS >

Name	Size	Type
 Generate Reports.bat	1 KB	Windows Batch File
 extract_nms_bases.py	26 KB	PY File
 extract_nms_bases_v8.py	26 KB	PY File
 ebaileytherogue's No Man's Sky_All Your Bases via Python Script.zip	547 KB	zip Archive
 README_NMS Base Information_Using These Files.pdf	556 KB	Chrome PDF Doc...
 save.hg.json	28,778 KB	JSON File

There will likely be two Python script files (.py) in the ZIP file you download, as shown above. They are identical. One does not contain the version number due to reasons outlined in changelog item [Changed Main Script Name to NOT Include Version Number and .BAT file to Reference This New Name](#).

The example is using a gamesave filename of save.hg.json, so rename your actual JSON file to match this prior to running the script. You can generate a JSON by using Goatfungus' app or the newer Vectorcmdr app, both named 'NMSSaveEditor'. You can probably use NomNom too, although I haven't tested that one.

I used a March 2026 build of Vectorcmdr and an older Goatfungus version to test and vet this script.

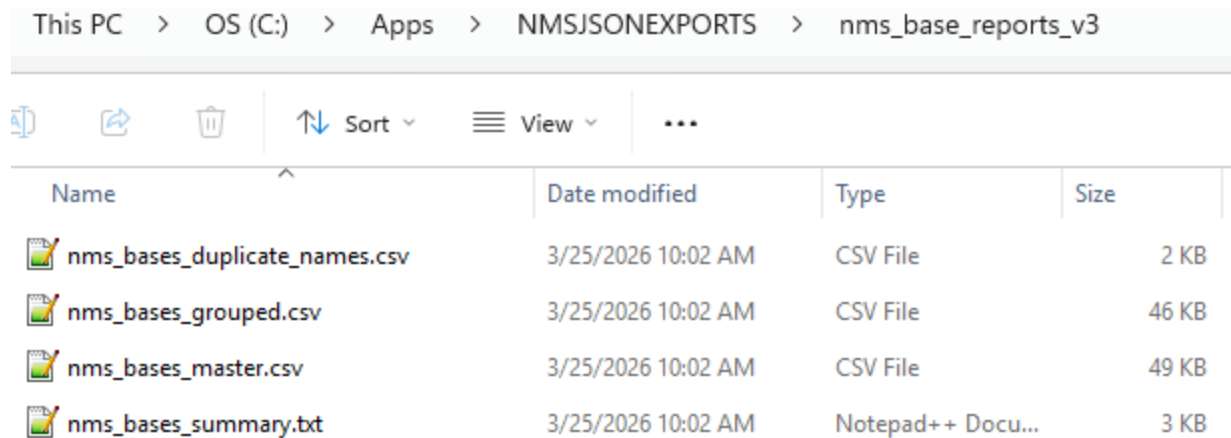
Once the files are in your folder of choice, simply double-click the .bat file to produce your reports. Example:

```
C:\WINDOWS\system32\cmd. x + v
C:\Apps\NMSJSONEXPORTS>python .\extract_nms_bases.py .\save.hg.json
C:\Apps\NMSJSONEXPORTS\extract_nms_bases.py:314: SyntaxWarning: "\" is an invalid escape sequence. Such sequences will
not work in the future. Did you mean "\\\"? A raw string is also an option.
"C:\Temp\Stuff" <-- invalid JSON, but some exporters still emit it
Standard JSON parse failed.
Reason: Invalid \escape: line 402772 column 15 (char 6952292)
Attempting automatic repair of invalid backslashes inside strings...
Master CSV written: C:\Apps\NMSJSONEXPORTS\nms_base_reports\nms_bases_master.csv
Grouped CSV written: C:\Apps\NMSJSONEXPORTS\nms_base_reports\nms_bases_grouped.csv
Duplicate report written: C:\Apps\NMSJSONEXPORTS\nms_base_reports\nms_bases_duplicate_names.csv
Summary written: C:\Apps\NMSJSONEXPORTS\nms_base_reports\nms_bases_summary.txt

Total unique teleporter-base rows: 203
Repeated base-name groups: 2

C:\Apps\NMSJSONEXPORTS>pause
Press any key to continue . . .
```

Ignore the syntax warning. It is the result of the script examining the JSON file and not totally digging it, but the warnings are simply warnings. The output is created successfully into a subfolder called nms_base_reports:



Name	Date modified	Type	Size
 nms_bases_duplicate_names.csv	3/25/2026 10:02 AM	CSV File	2 KB
 nms_bases_grouped.csv	3/25/2026 10:02 AM	CSV File	46 KB
 nms_bases_master.csv	3/25/2026 10:02 AM	CSV File	49 KB
 nms_bases_summary.txt	3/25/2026 10:02 AM	Notepad++ Docu...	3 KB

You can also run this manually if you'd prefer:

In Windows Explorer, type POWERSHELL in the address bar of the folder and press <ENTER>. A PowerShell window should open into that folder.

In the PowerShell window, copy and paste the following line, making sure to change the JSON filename to what your filename actually is:

```
python .\extract_nms_bases.py .\save.hg.json
```

Press <ENTER> to generate the files.

Dan Reams a.k.a. ebaleytherogue in No Man's Sky.



I saw (2026-04-02) that one of the bases in my reports files had a name older than the name as it appears in the online game. This was opposite to what I had imagined, as I'd assumed the local file would always have the latest and greatest values, with the online version only being updated either by manual base-upload or via some sort of timed synchronization.

Because I had renamed the base more than five days and 10-12 game sessions prior, and that sync had not already occurred, I think it safe to say that such synchronization only occurs via a manual base upload.

One might surmise that the online game is housing information, newer than the local gamesave, but that is a premature conclusion. Instead, having pored over the JSON file, I think the real issue is the changed name is present in the game-save, in some other location than the Teleporter list, and that the teleporter list -- which I use as the 'base name' -- is the one that gets updated after the manual upload. Bases are fickle things in the files, which is why I use the Teleporter list both for basenames and for "base" coordinates.

TL;DR - Simply make sure to upload a base after you rename it, and you can avoid this. I thought I always did, but I'd clearly missed at least one.

Change Log

I opt to include the change log items in this PDF to keep me honest. It forces me to update the documentation to reflect the changes I make to the script and its related files.

Sep 18, 2026

COSMOS UPDATE - More Base Types Added & Useless Reports Removed

Changes made:

- Added friendly **Base Type** columns.
- Made Planet-0 and base-computer warnings base-type aware.
- Prevented Planet-0 bases from borrowing unrelated teleporter metadata.
- Removed generation of two reports that no longer populated due to code clean-up:
 - `nms_bases_duplicate_names.csv`
 - `nms_bases_filtered_out.csv`
- The script also removes stale copies of those two files from the report folder.

Validated against real save files: E.g., all 402 bases were retained and only the master, grouped, and summary reports were produced.

Summary of COSMOS Changes:

I went through my **32.8 MB save**, the current `extract_nms_bases.py`, and ran the script against that save. I also specifically examined the raw `PersistentPlayerBases` records for the new Cosmos base types.

The good news is that **the core design of this script survived Cosmos remarkably well**. It found all **402 persistent-base records**, produced 402 report rows, filtered out **zero**, and found no duplicate-name groups. More importantly, the new Cosmos bases really are being stored in the same authoritative `BaseContext.PlayerStateData.PersistentPlayerBases` array I chose back in May. So I do **not** need to redesign the extraction architecture.

There are, however, a couple of Cosmos-specific improvements I thought to make:.

What Cosmos added to your save

The save now contains these `BaseType` values:

Raw save BaseType	Count	What it represents
<code>HomePlanetBase</code>	394	Conventional planetary bases

PlayerSpaceBase	4	Cosmos orbital/free-space/platform bases
PlayerSpaceStationBase	1	My claimed Cosmos space station
PlayerShipBase	2	Corvette/ship bases
FreighterBase	1	Freighter base

That agrees very nicely with Hello Games' description of Cosmos: orbital bases can now be built in empty space or on asteroid platforms, and stations can be claimed and customized.

And I can identify the new Cosmos records in the save quite cleanly.

For my test I had four **PlayerSpaceBase** records are:

- **Iffor-Ifri Colony**
- **Etbursl-Meph Outpost**
- **Neohit-Raseu Outpost**
- **Hiakar Base - Brad's Platform**

My claimed **PlayerSpaceStationBase** is:

- **Aisenn-Eudin-My Station Prime**

That one is particularly unmistakable. It contains **209 objects**, including 120 **^STACONNECT** objects, 33 vertical lights, station servers, shelving, etc. This is definitely the new claimed-station construction data.

So: **yes, the script is successfully finding the new Cosmos bases.**

The first thing I changed: report **BaseType**

My script previously read these records without actually putting **BaseType** into the reports.

Before Cosmos, that wasn't particularly important. Now it is.

I added a column:

Base Type

with friendly values such as:

- Planetary Base

- Orbital / Space Base
- Space Station
- Corvette / Ship Base
- Freighter Base

That made the reports considerably more useful now because, for example, `Planet = 0` has acquired an entirely legitimate meaning.

Cosmos has changed the significance of `PlanetIndex 0`

This is the biggest conceptual change.

All four `PlayerSpaceBase` records, the `PlayerSpaceStationBase`, both `PlayerShipBase` records, and the `FreighterBase` have:

`PlanetIndex = 0`

My old script consequently annotates them:

`PlanetIndex 0 (portal will error-correct to planet 1)`

That isn't really appropriate anymore.

For a Cosmos orbital base or space station, `PlanetIndex 0` is not an anomalous planetary address. **It's exactly what we'd expect for something that isn't on a planet.**

So I changed that note logic to something like:

- `HomePlanetBase` + `Planet 0` → retain the warning.
- Space/station/ship/freighter type + `Planet 0` → **no warning**.

The filtering logic itself is fine: none of these Cosmos records got filtered.

I found one actual matching problem

This is the one place where I would say the previous script was **not quite Cosmos-safe as-was**. Not unsafe to the gamesave, mind you -- that is never impacted; this script only READS data, and from the gamesave export file, never the gamesave directly -- but to the integrity of the reports.

My teleporter matching system uses:

Galaxy address + system + planet, followed by name/coordinate scoring.

Cosmos puts several fundamentally different objects in the same system at **`PlanetIndex 0`**. Consequently, several bases can share exactly the same address key.

For example, in one system the save contains:

- Iffor-Ifri Colony — PlayerSpaceBase
- Minimal Corvette 1 — PlayerShipBase
- My freighter — FreighterBase

The actual teleporter record contains **Iffor-Ifri Colony**.

My script correctly matches that teleporter to Iffor-Ifri Colony — **but it also assigns that same teleporter record to Minimal Corvette 1 and the freighter**, because they share the system/Planet-0 address.

Likewise, my:

Aisenn-Eudin-My Station Prime

space station currently gets supplemental teleporter metadata from:

Neohit-Raseu Outpost

because both are Planet-0 objects in the same system.

That explains something else I noticed: Cosmos 7.01 specifically changed the teleporter terminal to show screenshots of space-station bases, so Hello Games is actively integrating these new base types into the teleporter system.

This isn't corrupting the underlying base list or losing bases. **The authoritative base names and records are correct.** It's only my supplemental teleporter matching that can attach the wrong teleporter metadata to a Planet-0 object.

Don't worry. I fixed that in this updated script.

One more useful Cosmos distinction

The four orbital bases do **not** contain the traditional ^BUILDSAVE base-computer object my prior script looked for.

That's why they show as having no base computer.

But they aren't malformed. For example, the orbital bases contain things such as:

^SB_BEACON

and some contain:

^BUILDBEACON

Hello Games explicitly says Cosmos orbital bases use a new **Deep-Space Base Computer**.

So the old diagnostic:

No base computer object found

should not be applied indiscriminately to these new base types either.

This also explained my summary's rather startling:

Bases without base computer object: 292

Much of that count already reflected how my older minimal/foothold bases were constructed, but Cosmos gives us an additional reason not to treat ^BUILDSAVE as a universal validity test.

Bottom line

I would made **three Cosmos accommodations** rather than rewrite anything:

1. Added **Base Type** to the reports so Planetary / Orbital / Space Station / Ship / Freighter records are distinguishable.
2. Made the Planet-0 and "no base computer" notes **base-type aware**, so legitimate Cosmos space bases aren't flagged as suspicious.
3. Tightened **teleporter matching for Planet-0 records**, so an orbital base, Corvette, freighter, and space station in the same system can't borrow one another's teleporter metadata.

And, I removed the generation of:

nms_bases_duplicate_names.csv

and

nms_bases_filtered_out.csv

The evidence for removing those was pretty compelling in my gamesave: **0 duplicate-name groups and 0 filtered-out records**. The corresponding files contained nothing but their header row. Those reports contained no data for more than three revisions of this script over 10+ report-generation sessions, so I am confident they would never again generate data.

May 21, 2026

Major Update - Extraneous Bases That Were Listed from Temporary Data Removed from results.

I changed the logic to look at the first Persistent Bases list, in the JSON, as the primary source of truth versus the second Persistent Bases section. I now only use the second section for cross-reference information if needed.

I changed my methodology due to writing a base list resorter (the one on the internet did not function for me), and in doing so I learned more about the JSON structure.

Look at my other GitHub projects to find the base sorter as I've not yet uploaded it as of updating this change log.

More Date Fields Added to the Reports

The more date fields, the merrier. One can now sort on various base-related fields to more easily locate a base if one remembers nothing else about the base except for such a date. New fields:

```
Teleporter Order
Persistent Last Update Timestamp
Persistent Last Update Timestamp Raw
Earliest Object Timestamp
Earliest Object Timestamp Raw
Latest Object Timestamp
Latest Object Timestamp Raw
Base Computer Timestamp
Base Computer Timestamp Raw
Object Timestamp Count
```

Apr 8, 2026

Duplicate Bases Falsely Inflating the Base Count

In nms_bases_grouped I noticed, whilst scrolling, that some bases had the same Computer Coordinates. It looked like old base names were mixed in with later iterations of the same base. I created a pivot table, to group bases by Computer Coordinates, and discovered many duplicates. My total number of bases is thus falsely inflated.

I am currently going through the code to change to using a persistent base identify as the source of truth and then supplementing the winning records with information from the teleporter list. I think I can do that because the real base record lives in PersistentPlayerBases, which has the authoritative Name, LastUpdateTimestamp, and the Objects list containing the base computer ^BUILDSAVE.

This will take a while. WORKING ON THIS. I will publish this document to the folder but not bother including it in the ZIP file until I have a fix in place.

Mar 25, 2026

On-Planet Locations Added

I added two columns.

- Computer Coordinates - This was my attempt to provide 'Current Location' information to be used in one's scanner, or starship, or exocraft, or anywhere that has such functionality, to precisely locate a base. However, the information within the .JSON is not consistent for all bases, for reasons I won't get into here, so I added...
- Teleporter Location - This information IS consistent for every teleporter. So, unless you don't have a teleporter at your base, this should be all you need. However, I am leaving the Computer Coordinates value there a) as an interesting item, and b) for those edge cases where the value is both populated and you do not have a teleporter at that base.

Example Below:

G	H
Computer Coordinates	Teleporter Coordinates
-57.33, -79.92	-57.41, -80.06
	-55.22, -159.99
-57.33, -79.92	-24.15, -34.75
-27.88, +22.71	-27.88, +22.70
+49.95, +106.94	+49.96, +106.94
	-11.00, +45.96
	-15.71, -94.04
	+14.10, -156.00
	-45.52, -30.48
	-73.85, -30.36
+32.36, +127.63	+32.36, +127.65
	+57.72, -90.01

Teleporters Always Have Coordinates in the Teleporter System, so it's a good value to use.

Changed Reports Folder to always be "nms_base_reports"

During my testing I had included the script's version in the folder name so I would not get confused as to which version of reports I was reading. Now that the system is stable, changing the folder to a static value makes more sense.

Lines changed to:

```
default="nms_base_reports",  
help="Output directory (default: nms_base_reports)",
```

I changed all references to the reports folder, in the instructions portion of this document, to reflect this new name.

Changed Main Script Name to NOT Include Version Number and .BAT file to Reference This New Name

I did this so you would not have to rename or edit these files each time I release a new script version.

I am including both a versioned-named file and a non-version named file in the ZIP file, for your convenience.

Name

 extract_nms_bases.py

 extract_nms_bases_v8.py

In this example, the files are identical.

The instructional portions of this document have been edited to reflect this.

Changed the Called JSON Filename to save.hg.json from save8.hg.json

I did this so that the .bat file does not need to be changed each time your JSON filename changes. Instead, you will need to change the JSON filename to this new value prior to running the script. Reasoning: It's easier and less risky to change a JSON filename than open a batch file to change the called filename in there.

The instructional portions of this document have been edited to reflect this.